

TUNIX SOFTWARE AND EMBEDDED SYSTEMS

**T CORE MODULES: MEMORY, ERROR, & SELF-TEST
FORKS**

VERSION: 4.0.0

DOCUMENT REVISION: 2.0 | ARCHITECTURE: AVR / ATMEGA328P

1. Introduction

The Tunix Core Modules v4.0.0 aim to achieve a clean, type-safe, optimized, ready-to-use, and easy infrastructure management framework on Arduino Boards (ATMEGA328P) and other AVR-based microcontrollers. This version introduces expanded EEPROM operations with re-newed read/write functions and template support. Now EEPROM is accessed via Partitions created by user fore-hand or adjusted in capacity as the program's requirements. Supports up to 2+8 Partitions (Version, Settings, +8 Custom). All partitions could be mirrored with count functionality, for example 3 copies of RGB Struct.

2. Associated Files

The Tunix Core requires specific file pairs to be added to the include and src project folders. Memory and Error Manager Modules work together in order to interrupt device problems. Names and class names are changed in this version.

- **Memory Fork:** TMemoryManager.h, TMemoryManager.cpp
- **Error Fork:** TErrorManager.h, TErrorManager.cpp
- **Self-Test Fork:** TSelfTest.h, TSelfTest.cpp
- **Constants:** Constants.h

3. Module Specifications & Protocols

3.1. Memory Manager Fork

This version's new features are almost completely on the memory manager side. Now device reserves the first 64 byte for the Partition Table, full automatic table management is included in the update. Table holds Partitions' startAddress, size and count. Maximum partition is 10. Partitions with '0' size or '0' count counts as empty/deleted. A partition could store a struct with less size than the partition's itself.

- Template read/write functions allows flexible variable/file store system as long as memory is sufficient.
- If template's size is bigger than partition's allocation it returns an error to TErrorManager so device can take actions.

- In case of partition's size has to be changed `resizePartition` function seamlessly shift all the data forward or backwards automatically.

3.2. Error Manager Fork

Error Codes are updated due to extended errors, now instead of 'AB'; A as category and B as Specific Error Code. Now it 'ABC'; A as category, B as sub category and C as is Specific Error Code.

3.3. Self-Test Fork

Executes boot-level hardware integrity checks. This version does not include changes to Self-Test Fork.

- **RAM Integrity Test:** Scans the dynamic SRAM area using 0x5555 and 0xAAAA bitwise patterns to detect faulty registers. Restores original memory states post-test to prevent data corruption.
- **Free RAM Calculation:** Returns available SRAM in bytes via pointer arithmetic (`__heap_start` and `__brkval`).

3.3. Constants

Centralizes hardware pin definitions (Pins namespace), versioning structures (VersionInfo), system version declarations (SystemVersion namespace), and enum class `ErrorCode` definitions.

4. Standardized Error Codes Reference (Quick Look-up)

Use this reference table during runtime debugging and system auditing.

Error Code	Definition	Module Origin	Recommended Protocol
0	SYSTEM_OK	System	Continue Boot
101	FIRMWARE_VERSION_ERR OR	Memory Manager	Force Factory Reset

Error Code	Definition	Module Origin	Recommended Protocol
102	VERSION_NOT_SUPPORTED	Memory Manager	Force Factory Reset
103	DOWNGRADED_FIRMWARE	Memory Manager	Recommend Factory Reset
111	PROGRAM_LOOP_FAILURE	Main Loop	Reset Device
121	RAM_FAILURE	Self-Test	Halt System (Critical)
122	RAM_LOW	Self-Test	Device Specific Halt/Free Memory
131	EEPROM_FULL	Memory Manager	Halt System/Cancel EEPROM operation
132	ACCESS_VIOLATION	Memory Manager	Halt System, Update Firmware
133	BUFFER_OVERFLOW	Memory Manager	Cancel Writing
134	INTEGRITY_ERROR	Memory Manager	Re-Format EEPROM

5. Compatibility & Limitations

- Self-Test SRAM bitwise pattern scanning relies on AVR ATMEGA328P heap/stack memory pointers (`__heap_start`, `__brkval`, `RAMEND`). Non-AVR architectures (ESP32, STM32) require architecture conditional blocks (`#if defined(__AVR__)`).
- T Memory Manager might need specialized update path and pre-defined old data addresses due to new partition system TPT (Tunix Partition Table).
- New system's `BUFFER_OVERFLOW` error doesn't come with integrated cancel mechanism and has to be triggered manually.

6. EEPROM Memory Map (Address Layout)

The memory map below details the default ATMEGA328P EEPROM layout. Addresses are automatically calculated via TPT's address evaluation.

Address Range	Data Structure	Size (Bytes)	Description & Variables
0x00 - 0x59	Partition Data	60 Bytes	Holds 10 Partition's Data.
0x60 - 0x61	Device Signature	2 Byte	Holds an magic data to check if an corraption is present.
0x62 - 0x63	Transaction Flag	2 Byte	Holds either "Done" or "Progress" to safely execute updates.
0x64 - EOE	Dynamic Partitions	Dynamic	User defined, partitioned data section.

Warning: Do not manually overwrite addresses '0x00' to '0x63' using raw `EEPROM.write()` or `EEPROM.update()`. This will corrupt the Partition Table, might led to data corraption, un-intended device behavioir and triggering Error Handler. Device will reset it's Partition Table in case of corruption.

PS. To see the examples check the Extra Docs Folder.